

Competitive Programming

C++ 문제풀이 기반 교재

S0-A

C++ Basic Execution

v5.4 Learning Unit Part A-H 구조 기반

개념 설명, Worked Example, 실제 평가 문제, 학습자 제출 답안 수록

이 책의 목적과 사용법

이 문서는 한 번의 학습 세션을 단순히 기록하는 문서가 아니라, 같은 내용을 다시 처음부터 공부할 수 있도록 재구성한 **교재**다. 구성은 최신 작업 규범 v5.4의 Learning Unit 구조인 Part A-H를 따른다. Stage 0에 들어가기 전에 실제로 사용한 Initial Baseline 문제도 별도의 진단 문제 모음으로 수록했다.

이 책에서는 평가 판정, 난이도 판정, 통과 여부, 시간 기록, Review Debt와 같은 **개인 평가 결과를 실지 않는다**. 대신 학습 과정에서 실제로 제시된 평가 문제는 모두 보존한다. 일부 문제 뒤에는 당시 학습자가 직접 제출한 답안을 함께 실었다. 답안이 수록되지 않은 문제에는 해설이나 정답을 추가하지 않았다.

자료 보존 원칙

일부 초기 문제는 당시 문제 원문 전체나 학습자 답안 문장이 현재 기록에 완전하게 남아 있지 않다. 이 경우에는 확인 가능한 문제 핵심 또는 답안의 보존된 요지만 수록하고, 누락된 제약이나 예시를 임의로 만들어 채우지 않는다.

평가 문제를 풀 때의 기본 원칙

이 책에 실린 평가 문제를 다시 풀고 싶다면, 먼저 문제만 보고 독립적으로 해결한 뒤 답안이 수록된 문제만 비교하는 것을 권한다. 답안이 수록되지 않은 문제는 문제문만 남겨 두었으므로, 독립 재도전용으로 사용할 수 있다.

Contents

이 책의 목적과 사용법	ii
I Stage -1 — Initial Baseline	1
1 학습 시작점 진단 문제	2
1.1 Baseline의 역할	2
1.2 Foundation Check	2
1.2.1 FC-1 — 짝수의 개수와 합	2
1.2.2 FC-2 — 기하급수적으로 증가하는 내부 반복문	2
1.2.3 FC-3 — 대입 전에 발생하는 overflow	3
1.3 Baseline Problem A — Squares of a Sorted Array	3
1.4 Baseline Problem B — Remove Duplicates from Sorted Array	3
1.5 Baseline Problem C — Increasing Triplet Subsequence	4
II Stage 0 — C++ 문제풀이 기반	5
2 S0-A — C++ Basic Execution	6
2.1 Part A — 위치와 선수지식	6
2.1.1 현재 위치	6
2.1.2 왜 지금 배우는가	6
2.1.3 선수지식	7
2.2 Part B — 학습 목표	7
2.3 Part C — 개념과 원리	7
2.3.1 C1. 프로그램을 상태 전이로 바라보기	7
2.3.2 C2. cin과 입력 스트림	8
2.3.3 C3. 변수, 초기화, 대입	8
2.3.4 C4. 기본 자료형과 선택 기준	8
2.3.5 C5. 연산은 대입 전에 평가된다	9
2.3.6 C6. 조건문은 규칙의 우선순위를 코드로 옮긴다	9
2.3.7 C7. branch 안의 올바른 값을 뒤에서 덮어쓰지 않는가	9
2.3.8 C8. 반복문은 같은 코드의 반복이 아니라 상태 전이의 반복이다	10
2.3.9 C9. 상태 변수의 invariant	10
2.3.10 C10. 첫 원소를 특별 취급할 때의 위험	10
2.3.11 C11. 함수는 의미 있는 단위를 분리할 때 사용한다	10
2.3.12 C12. 시간복잡도: 반복 횟수와 연산 비용	11
2.3.13 C13. 추가 공간복잡도: 입력 크기에 따라 늘어나는 저장공간	11

2.3.14 C14. Edge case는 마지막 장식이 아니라 correctness 검증이다	11
2.3.15 C15. 손으로 trace하는 방법	12
2.3.16 C16. 제출 직전의 짧은 검증 루틴	12
2.4 Part D — Worked Example	12
2.5 Part E — 중간평가 문제	14
2.5.1 E-1. 양의 3의 배수 개수와 합	14
2.5.2 E-2. 부호/짝홀 변환 후 합	15
2.6 Part F — 최종 성취도 평가 문제	16
2.6.1 F-1. 부호 교대 구간	16
2.6.2 F-2. 잔고 회복 횟수와 최대 잔고	17
2.6.3 개념 보강: 초기 상태와 첫 transition	17
2.6.4 F-3. 기록 경신과 연속 상승	18
2.7 Part G — Adaptive Extra Problem	20
2.7.1 G-1. 누적 잔고의 최저점과 적자 연속 기간	20
2.8 Part H — 숙달 점검과 복습	21
2.8.1 H1. 개념 자기점검	21
2.8.2 H2. 구현 자기점검	22
2.8.3 H3. 이 Unit 이후에 연결되는 내용	22
2.8.4 H4. 문제 제시 형식에 대한 운영 원칙	23
A 빠른 C++ 문법 참고	24
A.1 입력과 출력	24
A.2 조건문	24
A.3 반복문	24
A.4 함수	24
B 평가 문제 색인	25

Part I

Stage -1 — Initial Baseline

Chapter 1

학습 시작점 진단 문제

1.1 Baseline의 역할

Initial Baseline은 학습 전에 현재 상태를 확인하기 위한 진단이다. 여기서는 간단한 입력/반복/조건 처리, 시간복잡도 판단, 정수 overflow, 배열 문제의 제약 이해, C++ container 문법, 작은 상태를 이용한 문제 해결 능력을 폭넓게 확인한다.

교재 관점에서 Baseline의 목적은 두 가지다. 첫째, 이후에 배울 개념이 왜 필요한지를 미리 드러낸다. 둘째, 같은 문제를 정답 암기로 해결하는 대신 새 문제에 개념을 전이할 수 있는지 확인하기 위한 출발점을 제공한다.

1.2 Foundation Check

1.2.1 FC-1 — 짝수의 개수와 합

Foundation Check 1

정수 N 과 이어서 N 개의 정수 x 가 주어진다. 짝수의 개수와 짝수들의 합을 출력한다.

구현 및 제출 조건

- C++로 완전한 프로그램을 작성한다.
- 시간복잡도와 추가 공간복잡도를 설명한다.

제약 조건

```
N <= 100000
-10000000000 <= x <= 10000000000
```

1.2.2 FC-2 — 기하급수적으로 증가하는 내부 반복문

Foundation Check 2

바깥 반복문이 N 회 실행된다. 각 바깥 반복마다 내부 변수 j 는 1에서 시작하고, 매 반복마다 $j *= 2$ 로 두 배가 되며 $j \leq N$ 인 동안 반복한다. 전체 시간복잡도를 구하고 이유를 설명하라.

학습자 답안 - 기록 보존 요지

내부 반복문은 $j = 1, 2, 4, 8, \dots$ 처럼 증가하므로 한 번의 바깥 반복에서 $O(\log N)$ 번 실행된다. 바깥 반복문은 N 번 실행되므로 전체 시간복잡도는

$$O(N \log N)$$

이다.

1.2.3 FC-3 — 대입 전에 발생하는 overflow**Foundation Check 3**

다음 코드는 안전한가? 문제가 있다면 원인을 설명하고 수정 방법을 제시하라.

```
1 int a = 10000000000;
2 int b = 10000000000;
3 long long result = a * b;
```

학습자 답안 - 기록 보존 요지

문제가 있다. `result`가 `long long`이어도 오른쪽의 `a * b`는 먼저 `int * int`로 계산되므로, 대입되기 전에 `overflow`가 발생할 수 있다. 연산 자체가 `long long`으로 이루어지도록 피연산자 중 하나 이상을 먼저 `long long`으로 승격해야 한다.

예를 들어 다음과 같이 쓸 수 있다.

```
1 long long result = 1LL * a * b;
```

1.3 Baseline Problem A — Squares of a Sorted Array**문제 A**

비내림차순으로 정렬된 정수 배열 `nums`가 주어진다. 각 원소를 제곱한 값들을 다시 비내림차순으로 정렬한 배열을 반환하라.

구현 및 제출 조건

- C++로 해결 방법을 작성한다.
- 시간복잡도와 공간복잡도를 설명한다.

1.4 Baseline Problem B — Remove Duplicates from Sorted Array**문제 B**

비내림차순으로 정렬된 정수 배열 `nums`가 주어진다. 중복 원소를 제거하되 별도의 결과 배열을 만드는 대신 **주어진 배열 자체의 앞부분**을 수정해야 한다. 서로 다른 원소의 개수를 k 라고 할 때, 배열의 처음 k 개 위치에는 각 고유 원소가 한 번씩 나타나야 한다. k 를 반환하라.

구현 및 제출 조건

- in-place 조건을 지켜야 한다.
- 시간복잡도와 추가 공간복잡도를 설명한다.

1.5 Baseline Problem C — Increasing Triplet Subsequence

문제 C

정수 배열 `nums`가 주어진다. 인덱스 $i < j < k$ 가 존재하여

$$\text{nums}[i] < \text{nums}[j] < \text{nums}[k]$$

를 만족하면 `true`, 그렇지 않으면 `false`를 반환하라.

구현 및 제출 조건

- 목표 시간복잡도: $O(N)$
- 목표 추가 공간복잡도: $O(1)$

Part II

Stage 0 — C++ 문제풀이 기반

Chapter 2

S0-A — C++ Basic Execution

2.1 Part A — 위치와 선수지식

2.1.1 현재 위치

Stage 0은 이후의 모든 알고리즘 학습에 앞서 C++로 정답 코드를 안정적으로 표현하기 위한 구간이다. 이번 Learning Unit인 S0-A는 다음 내용을 다룬다.

- 표준 입력과 출력
- 기본 자료형과 변수
- 조건문
- 반복문
- 함수
- scalar 상태를 이용한 기본 상태 전이

다음 Learning Unit인 S0-B에서는 `string`, `array`, `vector`, `pair`, `iterator`, `range-for`, `sort`, `reverse`, `comparator`, `reference`와 `const`를 다룬다. S0-C에서는 Big-O, 입력 크기와 실행 가능성, 정수 범위와 overflow를 더 체계적으로 다룬다.

2.1.2 왜 지금 배우는가

Competitive Programming에서는 알고리즘 아이디어만 맞는 것으로 충분하지 않다. 입력을 정확히 읽고, 현재 상태를 올바르게 유지하며, 조건을 정확한 순서로 적용하고, 마지막에 요구 형식대로 출력해야 실제 정답이 된다.

예를 들어 다음과 같은 실수는 고급 알고리즘을 몰라서 생기지 않는다.

- `N`을 읽지 않아 반복문이 한 번도 실행되지 않는다.
- 첫 원소만 특별 취급하면서 모든 원소에 적용되어야 할 규칙을 빠뜨린다.
- 조건문에서 값을 올바르게 설정한 뒤 바로 다음 줄에서 다시 덮어쓴다.
- 누적합을 `int`에 저장해 overflow가 발생한다.
- 현재 연속 길이를 언제 늘리고 언제 0 또는 1로 되돌려야 하는지 정의하지 않는다.

S0-A의 목표는 이런 기본 실행 오류를 줄이고, 이후의 자료구조와 알고리즘을 코드로 옮길 수 있는 안정적인 기반을 만드는 것이다.

2.1.3 선수지식

복잡한 알고리즘 지식은 필요하지 않다. 다음 정도를 알고 있으면 충분하다.

- C++ 프로그램이 `main()`에서 시작한다는 사실
- 정수와 실수를 변수에 저장할 수 있다는 사실
- 간단한 산술연산과 비교연산의 의미

2.2 Part B — 학습 목표

이 Unit을 마치면 다음 행동을 독립적으로 수행할 수 있어야 한다.

1. 입력 형식에 맞추어 `cin`으로 값을 빠짐없이 읽는다.
2. `int`, `long long`, `double`, `char`, `bool` 중 상황에 맞는 기본 자료형을 선택한다.
3. initialization과 assignment를 구분하고, 변수를 안전한 초기 상태로 시작한다.
4. `if`, `else if`, `else`로 규칙의 우선순위를 정확히 표현한다.
5. `for` 또는 `while`로 정확히 필요한 횟수만큼 입력을 처리한다.
6. 반복 중 유지할 상태 변수의 의미를 문장으로 정의한다.
7. 상태를 `extend`, `reset`, `start-new` 중 어떤 방식으로 바뀌어야 하는지 판단한다.
8. 필요할 때 의미 있는 helper function을 정의하고 호출한다.
9. 간단한 single-pass 프로그램의 시간복잡도와 추가 공간복잡도를 설명한다.
10. 최소 입력, 0, 부호 변화, reset 지점, 첫 transition 같은 edge case를 직접 구성해 검사한다.

2.3 Part C — 개념과 원리

2.3.1 C1. 프로그램을 상태 전이로 바라보기

S0-A에서 가장 중요한 관점은 프로그램을 다음 네 단계로 보는 것이다.

Input → State → Transition → Output

입력은 단순한 숫자의 나열이 아니다. 입력을 읽으면 프로그램 내부의 상태가 바뀌고, 그 상태가 다음 입력을 처리할 기준이 된다. 특히 한 번의 순회로 해결하는 문제에서는 **현재까지의 정보를 몇 개의 변수에 압축하여 유지하는 것이 핵심**이다.

예를 들어 지금까지 읽은 수의 합만 필요하다면 과거 원소를 모두 저장할 필요가 없다.

```

1 long long sum = 0;
2 for (int i = 0; i < N; ++i) {
3     long long x;
4     cin >> x;
5     sum += x;
6 }
```

이때 `sum`은 “지금까지 읽은 값들의 합”이라는 명확한 의미를 가진다.

2.3.2 C2. cin과 입력 스트림

`cin >> x`는 “사용자에게 한 줄을 입력받는다”라는 명령이라기보다, 표준 입력 스트림에서 `x`의 자료형으로 읽을 수 있는 다음 값을 가져오는 연산이다.

예를 들어

```
1 int a, b;
2 cin >> a;
3 cin >> b;
```

에서 입력이

```
10 20
```

이어도 되고,

```
10
20
```

이어도 된다. 정수 입력에서 공백, 탭, 줄바꿈은 모두 값 사이의 구분자로 작동한다.

따라서 Competitive Programming에서는 문제의 입력 형식을 보고 필요한 값을 정확한 순서로 읽는 것이 중요하다.

입력 제약

문제가 “첫 줄에 N , 다음에 N 개의 정수”라고 말한다면 프로그램은 먼저 N 을 읽고, 그 뒤 정확히 N 개의 값을 읽어야 한다. N 을 0으로 초기화하는 것과 `cin >> N;`으로 실제 입력을 읽는 것은 완전히 다른 일이다.

2.3.3 C3. 변수, 초기화, 대입

변수는 현재 상태를 저장하는 이름 붙은 공간이다.

```
1 int count = 0; // initialization
2 count = count + 1; // assignment
```

초기화는 변수가 처음 만들어질 때 시작값을 정한다. 대입은 이미 존재하는 변수의 값을 바꾼다. 반복문에서 쓰는 상태 변수는 초기값이 곧 알고리즘의 시작 상태이므로 특히 중요하다.

예를 들어 “현재까지 음수인 잔고가 연속된 길이”를 저장한다면 시작 전에는 아직 어떤 거래도 처리하지 않았으므로 자연스러운 초기값은 0이다.

2.3.4 C4. 기본 자료형과 선택 기준

S0-A에서 자주 쓰는 기본 자료형은 다음과 같다.

자료형	대표 용도	예시
<code>int</code>	작은 정수, 인덱스, 개수	N , loop index
<code>long long</code>	큰 정수, 누적합, 곱	최대 10^{14} 규모의 합
<code>double</code>	실수	비율, 평균
<code>char</code>	문자 하나	연산자, 알파벳
<code>bool</code>	참/거짓	조건 판정

자료형을 고를 때는 입력 하나의 범위만 보지 말고 **연산 결과의 최대 범위**를 봐야 한다. 예를 들어 $N \leq 10^5$, $|x_i| \leq 10^9$ 이면 누적합은 최대 약 10^{14} 이므로 `int`가 아니라 `long long`이 필요하다.

2.3.5 C5. 연산은 대입 전에 평가된다

다음 코드를 보자.

```
1 int a = 10000000000;
2 int b = 10000000000;
3 long long result = a * b;
```

`result`가 `long long`이라고 해서 안전하지 않다. `a * b`가 먼저 `int` 연산으로 계산된 뒤 그 결과가 `long long`에 대입되기 때문이다.

따라서 다음처럼 연산 전에 타입을 올려야 한다.

```
1 long long result = 1LL * a * b;
```

2.3.6 C6. 조건문은 규칙의 우선순위를 코드로 옮긴다

문제의 규칙이

$x < 0$ 이면 $-x$, 그렇지 않고 짝수이면 $x/2$, 그 외에는 $2x$

라면 코드도 같은 우선순위를 가져야 한다.

```
1 if (x < 0) {
2     ...
3 }
4 else if (x % 2 == 0) {
5     ...
6 }
7 else {
8     ...
9 }
```

`else if`는 앞 조건이 거짓일 때만 검사된다. 따라서 “음수이면서 짝수”인 값도 첫 번째 규칙에 의해 처리된다.

2.3.7 C7. branch 안의 올바른 값을 뒤에서 덮어쓰지 않는가

조건문에서 자주 생기는 오류는 올바른 값을 계산한 뒤 무조건 대입으로 다시 덮어쓰는 것이다.

```
1 if (condition) {
2     current = 1;
3 }
4 current = 0; // 위에서 정한 값을 무조건 덮어쓴다.
```

코드를 읽을 때는 “이 branch가 끝난 뒤 변수 값이 최종적으로 무엇인가?”를 확인해야 한다. 각 대입을 독립적으로 보는 대신, 한 iteration이 끝났을 때의 상태를 추적하는 습관이 필요하다.

2.3.8 C8. 반복문은 같은 코드의 반복이 아니라 상태 전이의 반복이다

for문을 “ N 번 실행”이라고만 이해하면 중요한 부분을 놓치기 쉽다. 더 좋은 관점은 다음과 같다.

각 iteration 시작 시 상태가 어떤 의미를 가지고 있고, 새 입력을 처리한 뒤 그 의미가 유지 되도록 상태를 갱신한다.

예를 들어 running balance 문제에서

B: 다음 거래를 처리하기 직전의 현재 잔고

라고 정의하면 초기값 $B=0$ 도 정상적인 상태다. 그러면 첫 거래 역시 이후 거래와 같은 규칙으로 처리해야 한다는 사실이 자연스럽게 보인다.

2.3.9 C9. 상태 변수의 invariant

Invariant는 반복 도중 항상 참이어야 하는 상태의 의미다. 복잡한 수학적 문장일 필요는 없다. S0-A에서는 다음처럼 간단한 문장도 강력하다.

예시 invariant

current: 현재 원소에서 끝나는, 조건을 만족하는 가장 긴 연속 구간의 길이.

이 정의가 있으면 새 원소를 만났을 때 세 가지 질문을 할 수 있다.

1. 기존 구간을 연장할 수 있는가? $\rightarrow \text{current} += 1$
2. 현재 원소가 조건을 깨는 값인가? $\rightarrow \text{current} = 0$
3. 기존 구간은 끊기지만 현재 원소에서 새 구간을 시작할 수 있는가? $\rightarrow \text{current} = 1$

이 세 경우를 구분하지 않으면 0, 같은 부호, 첫 원소 같은 경계에서 오류가 발생하기 쉽다.

2.3.10 C10. 첫 원소를 특별 취급할 때의 위험

첫 원소를 별도로 처리하는 것 자체가 나쁜 것은 아니다. 문제는 첫 원소를 따로 빼면서 **모든 원소에 적용되어야 하는 규칙까지 건너뛰는 것**이다.

예를 들어 초기 잔고가 0이고 “거래 직전 잔고가 0 이하, 거래 직후 양수”가 되는 순간을 세는 문제라면 첫 거래에서도 이 조건을 검사해야 한다. 첫 거래를

```
1 if (i == 1) {
2     balance = x;
3 }
```

처럼 통째로 별도 처리하면 첫 transition에 적용되어야 할 규칙이 빠질 수 있다.

더 안전한 설계는 가능한 한 모든 iteration에 같은 transition을 적용하고, 정말 필요한 상태만 따로 초기화하는 것이다.

2.3.11 C11. 함수는 의미 있는 단위를 분리할 때 사용한다

함수의 기본 형태는 다음과 같다.

```
1 return_type function_name(parameters) {
2     // work
3     return value;
4 }
```

함수는 다음과 같은 경우에 유용하다.

- 같은 로직을 여러 번 재사용할 때
- 하나의 의미 있는 판정이나 변환을 이름으로 표현하고 싶을 때
- 재귀, DFS, comparator처럼 독립된 동작이 자연스러울 때
- `main()`이 지나치게 길어져 기능별 분리가 필요할 때

하지만 $x > y$ 한 줄을 단지 “함수를 하나 만들어야 한다”는 조건 때문에 함수로 감싸는 것은 좋은 추상화가 아니다.

```
1 bool greaterThan(long long x, long long y) {
2     return x > y;
3 }
```

이 함수가 틀린 것은 아니지만, 코드의 의미를 크게 개선하지 않는다.

함수 사용 원칙

함수는 개수가 중요한 것이 아니라 **의미 있는 책임을 분리하는**가가 중요하다. 이후 문제에서는 함수 정의 자체가 학습목표이거나 문제 구조상 함수 분리가 자연스러운 경우에만 별도 함수 사용을 강제한다.

2.3.12 C12. 시간복잡도: 반복 횟수와 연산 비용

한 번의 `for`문이 N 번 실행되고 각 iteration에서 상수 시간 연산만 수행한다면 전체 시간복잡도는 $O(N)$ 이다.

```
1 for (int i = 0; i < N; ++i) {
2     cin >> x;
3     sum += x;
4 }
```

반면 내부 반복 횟수가 1, 2, 4, 8, ... 처럼 증가한다면 내부 반복은 $O(\log N)$ 이고, 이것이 바깥 N 회 반복 안에 있다면 $O(N \log N)$ 이 된다.

중요한 것은 “코드 줄 수”가 아니라 각 연산이 몇 번 실행되는가를 보는 것이다.

2.3.13 C13. 추가 공간복잡도: 입력 크기에 따라 늘어나는 저장공간

`int`, `long long` 변수 몇 개만 사용하는 것은 N 이 커져도 변수 개수가 늘어나지 않으므로 $O(1)$ 추가 공간이다.

반면 입력 N 개를 `vector`에 모두 저장하면 추가 공간은 $O(N)$ 이다. 과거 원소를 다시 사용할 필요가 없다면 입력을 읽는 즉시 처리하여 $O(1)$ 공간으로 줄일 수 있다.

2.3.14 C14. Edge case는 마지막 장식이 아니라 correctness 검증이다

좋은 코드는 예시 입력 하나를 통과하는 코드가 아니라, 문제의 모든 조건에서 규칙이 유지되는 코드다. 최종 제출 전에 최소한 다음 유형을 직접 확인한다.

- 최소 크기: $N = 1$
- 조건을 전혀 만족하지 않는 경우: 모두 0, 모두 같은 부호 등
- 조건을 계속 만족하는 경우: 끝까지 증가, 끝까지 부호 교대 등

- **reset 경계:** 유효 구간 중간에 0 또는 규칙 위반 값이 등장
- **첫 transition:** 초기 상태에서 첫 입력으로 넘어갈 때 조건이 발생하는가
- **수치 경계:** $\pm 10^9$ 등 제약의 끝값

2.3.15 C15. 손으로 trace하는 방법

상태 전이 문제에서 가장 효과적인 디버깅 방법 중 하나는 표를 만드는 것이다.

예를 들어 초기 잔고가 0이고 입력이 5 -10 7이라고 하자. 잔고가 0 이하에서 양수로 바뀌는 횟수를 센다면 다음처럼 추적할 수 있다.

입력	처리 전 잔고	처리 후 잔고	전환 횟수
5	0	5	1
-10	5	-5	1
7	-5	2	2

이 표는 첫 거래도 정상적인 transition이라는 점을 분명하게 보여준다.

2.3.16 C16. 제출 직전의 짧은 검증 루틴

코드를 다 작성한 직후에는 다음 순서로 확인한다.

1. 입력 개수와 반복 횟수가 맞는가?
2. 각 상태 변수의 의미를 한 문장으로 설명할 수 있는가?
3. 첫 iteration에도 적용되어야 할 규칙을 우회하지 않았는가?
4. 조건문의 한 branch에서 만든 값을 뒤에서 덮어쓰지 않는가?
5. $N = 1$ 을 직접 넣었는가?
6. reset을 일으키는 값을 넣었는가?
7. 누적합이나 곱의 최대 범위가 자료형 안에 들어가는가?
8. 요구 시간/공간복잡도를 만족하는가?

2.4 Part D — Worked Example

D1. 문제

Worked Example — 양수의 합

정수 N 과 이어서 N 개의 정수가 주어진다. 양수인 값만 모두 더해 합을 출력하라.

구현 조건

- 입력을 한 번씩 처리하는 방법을 우선 생각한다.
- 합이 커질 수 있으므로 안전한 자료형을 선택한다.

D2. Brute Force

가장 직접적인 방법은 모든 값을 배열에 저장한 뒤, 다시 배열을 순회하며 양수만 더하는 것이다. 이 방법은 정답을 구할 수 있다.

D3. Complexity

값 N 개를 저장하고 다시 한 번 순회하면 시간복잡도는 $O(N)$ 이고 추가 공간복잡도는 $O(N)$ 이다.

D4. Bottleneck

시간은 이미 선형이므로 큰 문제가 없다. 하지만 이 문제에서는 과거의 모든 원소를 나중에 다시 불필요가 없다. 따라서 $O(N)$ 저장공간은 불필요하다.

D5. 핵심 관찰

현재 입력 x 가 양수인지 여부만 알면, 그 자리에서 바로 합에 반영할 수 있다. 과거 원소 자체는 더 이상 필요하지 않다.

D6. 알고리즘 설계

1. N 을 읽는다.
2. $sum = 0$ 으로 시작한다.
3. N 번 반복하며 x 를 하나 읽는다.
4. $x > 0$ 이면 $sum += x$ 를 수행한다.
5. 반복이 끝나면 sum 을 출력한다.

D7. 정확성

각 입력은 정확히 한 번 처리된다. 양수인 입력은 한 번씩 sum 에 더해지고, 0 이나 음수는 더해지지 않는다. 따라서 반복 종료 후 sum 은 정확히 모든 양수의 합이다.

D8. C++ 구현

```
1 #include <iostream>
2 using namespace std;
3
4 int main() {
5     int N;
6     cin >> N;
7
8     long long sum = 0;
9     for (int i = 0; i < N; ++i) {
10         long long x;
11         cin >> x;
12         if (x > 0) {
13             sum += x;
14         }
15     }
```

```

15     }
16
17     cout << sum << '\n';
18 }

```

D9. Trace

입력이 -2, 5, 0, 7이라면 sum은 다음처럼 변한다.

현재 입력	sum
-2	0
5	5
0	5
7	12

D10. Edge Cases

- $N = 1$, 입력이 양수
- $N = 1$, 입력이 0
- 모든 값이 음수
- 모든 값이 양수
- 큰 양수들이 반복되어 합이 int 범위를 넘는 경우

2.5 Part E — 중간평가 문제

2.5.1 E-1. 양의 3의 배수 개수와 합

중간평가 문제 E-1

N 개의 정수를 읽고, 양수이면서 3의 배수인 값들의 개수와 합을 출력하라.

구현 및 제출 조건

- 당시 문제에서는 `main()` 이외의 helper function을 직접 정의하고 실제로 사용하도록 요구했다.

원문 보존 상태

이 문제의 당시 전체 제약 조건과 sample은 현재 기록에 완전하게 남아 있지 않다. 따라서 확인 가능한 핵심 요구사항만 수록했다.

2.5.2 E-2. 부호/짝홀 변환 후 합

중간평가 문제 E-2

정수 N 과 이어서 N 개의 정수 x 가 주어진다. 각 x 를 다음 규칙으로 변환한 뒤 모든 변환값의 합을 출력하라.

- $x < 0$ 이면 $-x$
- 그렇지 않고 x 가 짝수이면 $x/2$
- 그 외에는 $2x$

구현 및 제출 조건

- C++17 또는 C++20
- `main()` 이외의 helper function을 하나 이상 정의하고 실제로 사용한다.
- 시간복잡도와 공간복잡도를 설명한다.

제약 조건

```
N <= 100000
-1000000 <= x <= 1000000
```

학습자 제출 답안

```
1 #include <iostream>
2 using namespace std;
3
4 long long func(long long x) {
5     long long result = 0;
6     if (x < 0) {
7         result = -x;
8     }
9     else if (x % 2 == 0) {
10        result = x / 2;
11    }
12    else {
13        result = 2 * x;
14    }
15    return result;
16 }
17
18 int main()
19 {
20     int N = 0;
21     cin >> N;
22     long long sum = 0;
23     for (int i = 1; i <= N; i += 1) {
24         long long x = 0;
25         cin >> x;
26         sum += func(x);
```

```

27     }
28     cout << sum;
29 }

```

함수의 역할: func는 입력된 하나의 정수에 문제의 변환 규칙을 적용한 값을 반환한다.
시간복잡도: 반복문이 N 번 실행되고 각 iteration의 연산은 $O(1)$ 이므로 총 $O(N)$ 이다.
공간복잡도: 입력 전체를 저장하지 않고 몇 개의 scalar 변수만 사용하므로 $O(1)$ 이다.
당시 직접 확인한 edge case: 없음.

2.6 Part F — 최종 성취도 평가 문제

2.6.1 F-1. 부호 교대 구간

최종평가 문제 F-1

정수 N 과 이어서 N 개의 정수가 주어진다. 다음 두 조건을 모두 만족하는 **연속 구간**의 최대 길이를 출력하라.

1. 구간 안에 0이 없다.
2. 인접한 두 수의 부호가 항상 반대다.

구현 및 제출 조건

- C++17 또는 C++20
- 당시 문제에서는 `main()` 이외의 함수를 최소 하나 정의하고 실제로 사용하도록 요구했다.
- 요구 시간복잡도: $O(N)$
- 요구 추가 공간복잡도: $O(1)$
- 완성 코드, 함수 역할, 시간복잡도, 공간복잡도, 직접 확인한 edge case를 제출한다.

제약 조건

```

1 <= N <= 100000
-10000000000 <= x <= 10000000000

```

예시 입력

```

8
1 -2 3 4 -5 6 0 -7

```

예시 출력

```

3

```

2.6.2 F-2. 잔고 회복 횟수와 최대 잔고

최종평가 문제 F-2

초기 잔고는 0이다. 정수 N 과 이어서 N 개의 정수 x 가 주어진다. 각 x 를 순서대로 잔고에 더한다. 각 거래를 처리할 때,

- 거래 직전 잔고가 0 이하이고
- 거래 직후 잔고가 양수라면

이를 **recovery** 1회라고 정의한다.

모든 거래를 처리한 뒤 다음 두 값을 출력하라.

1. recovery가 발생한 총 횟수
2. 모든 거래 직후 잔고들 중 가장 큰 값

초기 잔고 0은 두 번째 값의 후보에 포함하지 않는다.

구현 및 제출 조건

- C++17 또는 C++20
- 당시 문제에서는 `main()` 이외의 함수를 최소 하나 정의하고 실제로 사용하도록 요구했다.
- 요구 시간복잡도: $O(N)$
- 요구 추가 공간복잡도: $O(1)$
- 완성 코드, 함수 역할, 시간복잡도, 공간복잡도, 직접 확인한 edge case를 제출한다.

제약 조건

```
1 <= N <= 100000
-10000000000 <= x <= 10000000000
```

예시 입력

```
7
-3 5 -4 1 2 -10 12
```

거래 후 잔고는 -3, 2, -2, -1, 1, -9, 3이다.

예시 출력

```
3 3
```

2.6.3 개념 보강: 초기 상태와 첫 transition

F-2와 같은 문제에서는 초기 상태를 반복문 밖의 예외가 아니라 상태 머신의 시작점으로 보는 것이 중요하다.

초기 잔고가 0이고 입력이 5 -10 7인 경우를 다시 보자.

거래	처리 전 잔고	처리 후 잔고	recovery 누적
5	0	5	1
-10	5	-5	1
7	-5	2	2

핵심은 첫 거래도 “처리 전 상태 → 처리 후 상태”라는 동일한 transition이라는 점이다. 첫 iteration 을 별도로 처리하더라도 recovery 판정 자체를 생략해서는 안 된다.

2.6.4 F-3. 기록 경신과 연속 상승

최종평가 문제 F-3

정수 N 과 점수 $a_1, a_2, \dots, a_N$ 이 주어진다. 다음 두 값을 구하라.

1. **기록 경신 횟수**: a_i 가 a_i 이전에 등장한 모든 점수보다 엄격하게 크다면 기록 경신이다. 첫 번째 점수 a_1 은 항상 기록 경신으로 센다.
2. **가장 긴 연속 상승 구간의 길이**: 연속 구간 $a_l, a_{l+1}, \dots, a_r$ 이

$$a_l < a_{l+1} < \dots < a_r$$

를 만족할 때 그 길이 $r - l + 1$ 의 최댓값을 구한다.

기록 경신 횟수와 가장 긴 연속 상승 구간의 길이를 공백으로 구분하여 출력하라.

구현 및 제출 조건

- C++17 또는 C++20
- 당시 문제에서는 `main()` 이외의 함수를 최소 하나 정의하고 실제로 사용하도록 요구했다.
- 요구 시간복잡도: $O(N)$
- 요구 추가 공간복잡도: $O(1)$
- 완성 코드, 함수 역할, 시간복잡도, 공간복잡도, 직접 확인한 edge case를 제출한다.
- Visual Studio의 compile/run, breakpoint, step 실행, 변수 확인을 사용할 수 있다.

제약 조건

```
1 <= N <= 100000
-10000000000 <= a_i <= 10000000000
```

예시 입력

```
8
3 5 4 6 7 2 3 4
```

예시 출력

```
4 3
```

학습자 제출 답안

```

1 #include <iostream>
2 using namespace std;
3
4 bool func(long long x, long long y) {
5     if (x > y) {
6         return true;
7     }
8     else {
9         return false;
10    }
11 }
12
13 int main()
14 {
15     int N = 0;
16     int r = 0;
17     int L1 = 0;
18     int L2 = 1;
19     long long tempM = -1000000001;
20     long long temp = -1000000001;
21     cin >> N;
22     for (int i = 1; i <= N; i += 1) {
23         long long a = 0;
24         cin >> a;
25         if (func(a, tempM)) {
26             r += 1;
27             L1 += 1;
28             tempM = a;
29         }
30         else if (!func(a, tempM) && func(a, temp)) {
31             L1 += 1;
32         }
33         else if (!func(a, tempM) && !func(a, temp)) {
34             L1 = 1;
35         }
36         if (L1 > L2) {
37             L2 = L1;
38         }
39         temp = a;
40     }
41     cout << r << " " << L2;
42 }

```

함수의 역할: func는 두 정수의 대소를 비교한다.

시간복잡도: N 번의 반복을 수행하고 각 iteration에서 상수 시간 연산만 하므로 $O(N)$ 이다.

공간복잡도: 입력 크기와 무관한 수의 scalar 변수만 사용하므로 $O(1)$ 이다.

직접 확인한 edge case: $N = 1$ 일 때 예상 출력은 1 1.

2.7 Part G — Adaptive Extra Problem

2.7.1 G-1. 누적 잔고의 최저점과 적자 연속 기간

Adaptive Extra Problem G-1

초기 잔고는 0이다. 정수 N 과 변화량 $x_1, x_2, \dots, x_N$ 이 주어진다. 각 변화량을 순서대로 현재 잔고에 더한다. i 번째 변화량을 처리한 직후의 잔고를 B_i 라고 한다.

다음 두 값을 출력하라.

1. 모든 B_i 중 가장 작은 값
2. $B_i < 0$ 인 상태가 연속해서 유지된 가장 긴 기간의 길이

초기 잔고 0은 첫 번째 값의 후보에도, 두 번째 값의 기간에도 포함하지 않는다.

구현 및 제출 조건

- C++17 또는 C++20
- 당시 문제에서는 `main()` 이외의 함수를 최소 하나 정의하고 실제로 사용하도록 요구했다.
- 요구 시간복잡도: $O(N)$
- 요구 추가 공간복잡도: $O(1)$
- 완성 코드, 함수 역할, 시간복잡도, 공간복잡도, 직접 확인한 edge case를 제출한다.
- Visual Studio debugger 사용이 허용된다.

제약 조건

```
1 <= N <= 100000
-10000000000 <= x_i <= 10000000000
```

예시 입력

```
7
-3 1 -1 5 -4 -2 6
```

각 변화 후 잔고는 -3, -2, -3, 2, -2, -4, 2이다.

예시 출력

```
-4 3
```

학습자 제출 답안

```
1 #include <iostream>
2 using namespace std;
3
4 bool func(long long x, long long y) {
5     if (x > y) {
6         return true;
```

```

7     }
8     else {
9         return false;
10    }
11 }
12
13 int main()
14 {
15     long long B = 0;
16     long long b = 1000000001;
17     int N = 0;
18     int r = 0;
19     int L1 = 0;
20     int L2 = 0;
21     cin >> N;
22     for (int i = 1; i <= N; i += 1) {
23         long long x = 0;
24         cin >> x;
25         B += x;
26         if (func(b, B)) {
27             b = B;
28         }
29         if (B < 0) {
30             L1 += 1;
31         }
32         else {
33             L1 = 0;
34         }
35         if (L1 > L2) {
36             L2 = L1;
37         }
38     }
39     cout << b << " " << L2;
40 }

```

함수의 역할: func는 두 값의 대소를 비교한다.

시간복잡도: 반복문이 N 번 실행되고 각 iteration에서 상수 시간 연산만 수행하므로 $O(N)$ 이다.

공간복잡도: 입력 크기와 무관한 수의 변수만 사용하므로 $O(1)$ 이다.

직접 확인한 edge case: $N = 1$ 일 때 $x = -123$ 이면 -123 1, $x = 123$ 이면 123 0을 예상했다.

2.8 Part H — 숙달 점검과 복습

이 교재판에서는 개인 평가 판정을 기록하지 않는다. 대신 S0-A의 핵심 능력을 다시 점검할 수 있는 질문과 복습 기준을 제공한다.

2.8.1 H1. 개념 자기점검

다음 질문에 자료 없이 답할 수 있는지 확인하라.

1. `cin >> a >> b;`는 입력 스트림에서 어떤 순서로 값을 읽는가?

2. `long long result = a * b;`에서 `a, b`가 `int`라면 왜 overflow가 대입 전에 발생할 수 있는가?
3. `if / else if / else`와 서로 독립적인 여러 개의 `if`는 어떤 차이가 있는가?
4. 반복문에서 “현재 상태 변수의 invariant”를 문장으로 정의하면 어떤 장점이 있는가?
5. 첫 원소를 별도 처리하는 것이 위험할 수 있는 이유는 무엇인가?
6. 연속 구간 문제에서 현재 길이를 0으로 reset하는 경우와 1로 start-new하는 경우는 어떻게 다른가?
7. 입력 N 개를 저장하지 않고도 한 번의 순회로 해결할 수 있는 문제의 특징은 무엇인가?
8. 함수가 의미 있는 추상화가 되는 경우와 단순 wrapper에 불과한 경우를 구분할 수 있는가?
9. $N = 10^5$, $|x_i| \leq 10^9$ 인 누적합 문제에서 왜 `long long`이 필요한가?

2.8.2 H2. 구현 자기점검

코드 작성 후 체크리스트

- ☐ 문제에서 요구한 모든 입력을 읽었는가?
- ☐ 반복 횟수가 정확한가?
- ☐ 현재 상태 변수의 의미를 한 문장으로 설명할 수 있는가?
- ☐ 첫 iteration이 일반 규칙을 우회하지 않는가?
- ☐ branch 안의 값이 뒤에서 덮어써지지 않는가?
- ☐ $N = 1$ 을 직접 테스트했는가?
- ☐ reset을 유발하는 입력을 테스트했는가?
- ☐ 모든 값이 조건을 만족하는 경우를 테스트했는가?
- ☐ 모든 값이 조건을 만족하지 않는 경우를 테스트했는가?
- ☐ 합/곱의 최대 크기를 계산해 자료형을 선택했는가?
- ☐ 시간복잡도와 추가 공간복잡도를 설명할 수 있는가?

2.8.3 H3. 이 Unit 이후에 연결되는 내용

S0-A에서 익힌 실행 능력은 다음 주제에서 그대로 사용된다.

- **S0-B: vector, string, array, pair, iterator, sort** 등 container/STL을 사용하더라도 상태 전이와 edge-case 검증은 동일하게 필요하다.
- **S0-C:** 지금까지 직관적으로 다룬 $O(N)$, $O(N \log N)$, 정수 범위와 overflow를 더 체계적으로 정리한다.
- **이후 알고리즘:** two pointers, sliding window, DP, graph traversal에서도 결국 반복문의 각 시점에 어떤 상태가 유지되는지가 correctness의 핵심이 된다.

2.8.4 H4. 문제 제시 형식에 대한 운영 원칙

앞으로 평가 문제를 제시할 때는 학습자가 답안 작성 전에 반드시 확인해야 하는 내용을 먼저 볼 수 있도록 다음 순서를 사용한다.

1. 문제 설명
2. 구현 및 제출 조건
3. 제약 조건
4. 예시 입력/출력

또한 별도 함수 사용은 모든 문제에 일률적으로 강제하지 않는다. 함수 정의 자체가 학습 목표이거나 문제 구조상 기능 분리가 자연스러운 경우에만 요구한다.

Appendix A

빠른 C++ 문법 참고

A.1 입력과 출력

```
1 int N;  
2 long long x;  
3 cin >> N >> x;  
4 cout << N << ' ' << x << '\n';
```

A.2 조건문

```
1 if (condition1) {  
2     // ...  
3 }  
4 else if (condition2) {  
5     // ...  
6 }  
7 else {  
8     // ...  
9 }
```

A.3 반복문

```
1 for (int i = 0; i < N; ++i) {  
2     // ...  
3 }
```

A.4 함수

```
1 long long transform(long long x) {  
2     return x * 2;  
3 }
```

Appendix B

평가 문제 색인

ID	문제
FC-1	짝수의 개수와 합
FC-2	기하급수적으로 증가하는 내부 반복문의 시간복잡도
FC-3	대입 전에 발생하는 정수 overflow
Baseline A	Squares of a Sorted Array
Baseline B	Remove Duplicates from Sorted Array
Baseline C	Increasing Triplet Subsequence
E-1	양의 3의 배수 개수와 합
E-2	부호/짝홀 변환 후 합
F-1	부호 교대 구간
F-2	잔고 회복 횟수와 최대 잔고
F-3	기록 경신과 연속 상승
G-1	누적 잔고의 최저점과 적자 연속 기간